

Observability and Debugging of Agents

Tracing Agent Trajectories · Evaluating Decision Points

Monitoring Agent Behavior in Production

Akshita Vijayakumar

Agenda

01 Motivation & Background

Why agents are hard to debug

02 Agent Trajectories & Tracing

Recording execution paths

03 Evaluating Decision Points

Where does the agent fail?

04 Tools & Frameworks

LangSmith, Phoenix, Weave, Langfuse

05 Comparison

Side-by-side evaluation

06 Prototype Demo

Agent Trace Debugger — live walk-through

07 Conclusion & Future Work

Takeaways and open challenges

What Is a Tool-Using AI Agent?

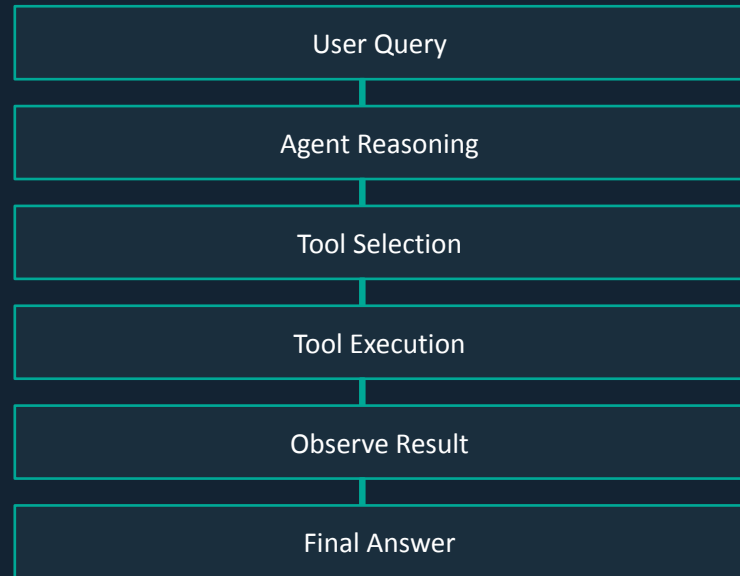
Definition

An AI agent is a system that perceives its environment, reasons about a goal, and takes actions through external tools to produce a result.

Unlike a chatbot that generates a single response, an agent:

- Plans multi-step workflows
- Selects and calls external tools
- Observes tool results and adapts
- Iterates until the goal is reached

Agent Execution Flow



Why Is Debugging Agents So Difficult?

Hidden Reasoning

Internal decisions are not visible. A wrong answer reveals nothing about where it went wrong.

Multi-Step Failures

A wrong tool selection early in the execution silently corrupts all subsequent steps, producing a plausible but incorrect result.

Non-Determinism

Same input, different output. Bugs appear intermittently and resist reproduction.

Tool Interaction Errors

Correct tool, wrong arguments - executes without error, returns wrong data.

Key insight: Evaluating only the final answer is insufficient — the full execution trajectory must be observed.

What Is Observability?

LOGS

Timestamped records of every tool call, decision, and error

TRACES

Full execution path from user query to final answer

METRICS

Numerical signals: success rate, latency, error rate, cost

Traditional Software vs. Agentic AI

Traditional: Logs reveal what happened, Traces show the path, Metrics quantify health.

Agentic AI adds: decision-point visibility, tool-call auditing, reasoning transparency, and constraint validation.

Agent Trajectories

An agent trajectory is the full record of every action an agent takes — from receiving a query to producing a final answer

1

User Query

The original input and reference point for all checks

2

Query Classification

Agent identifies intent and selects a reasoning path

3

Tool Selection

Agent chooses which tool to invoke

4

Tool Input Generation

Agent constructs tool arguments - a major failure point

5

Tool Execution

External tool runs and returns output or error

6

Output Observation

Agent reads and interprets the tool result

7

Final Answer

Final response - must be grounded in retrieved evidence

Trace Logging — Recording Every Step

What a trace record captures:

run_id	Unique identifier for the run
step_id	Sequential step counter
timestamp	Execution time of the step
event_type	user_query / tool_call / final_answer ...
agent_decision	What the agent decided to do
tool_name	Which tool was called
tool_input	Arguments sent to the tool
tool_output	Result returned by the tool
error_message	Error details if the step failed

Example trace record (JSON):

```
{
  "run_id": "run_679448c3",
  "step_id": 5,
  "event_type": "tool_call",
  "agent_decision":
    "execute_structured_search",
  "tool_name": "structured_search",
  "tool_input": {
    "city": "Munich",
    "max_price": 200
  },
  "tool_output": [...],
  "status": "success"
}
```

Evaluating Agent Decision Points

Not all steps carry equal risk. These are the points where failures originate

Was a tool needed?

Medium

Unnecessary tool calls add latency and introduce risk

Was the right tool selected?

High

Wrong tool = incorrect output regardless of later steps

Were tool arguments correct?

Critical

Correct tool, wrong input — the most common silent failure

Was the tool result used?

High

Agent may ignore retrieved evidence and hallucinate instead

Does the answer match constraints?

Critical

Final answer may violate a constraint from the original query

Did the agent stop at the right time?

Medium

Premature stop or infinite retry both degrade output quality

Common Failure Modes in Tool-Using Agents

Wrong Tool Selection

User asks for a document summary → agent calls calculator

Incorrect Tool Arguments

User: under €120 → agent sends max_price = 200

Missing Structured Fields

Hotel search without city → returns no results silently

Unsupported Query

Agent finds no matching tool and returns fallback response

Tool Execution Error

External API fails → agent propagates error to final answer

Repeated Tool Calls

Agent retries the same tool call repeatedly without adapting arguments

Ignored Evidence

Document retrieved but agent ignores it and hallucinates

Constraint Violation

Final answer recommends hotel at €180 when limit was €120

From Distributed Tracing to Agent Observability



LangSmith

Source: LangChain Inc. LangSmith Documentation. smith.langchain.com, 2024. [9]

PRODUCTION
READY

Full LLM chain tracing

Auto-instruments LangChain agents, captures prompts, outputs and tool calls at every node

Visual run explorer

Shows execution trees, token counts, latency and tool call details

Dataset & evaluation

Build ground-truth datasets, score agent outputs at scale

Playground & feedback

Replay any logged run with modified inputs; collect human feedback inline

Limitation: Tightly coupled to LangChain; complex setup for custom agents; paid for team features

Arize Phoenix

Source: Arize AI. Phoenix Observability Platform. phoenix.arize.com, 2024. [10]

OPEN
SOURCE

OpenTelemetry-native tracing

Ingests OTEL spans - works with any framework, not just LangChain

Evaluation pipelines

Built-in evaluators for hallucination, Q&A correctness and custom criteria

Embedding analytics

Visualizes embedding drift and cluster analysis for retrieval agents

Prompt playground

Test prompt variations against stored traces without re-running the agent.

Limitation: Heavier infrastructure setup, embedding analytics add complexity for simple debugging tasks

Weights & Biases — Weave

Source: *Weights & Biases. Weave Documentation. weave.wandb.ai, 2024. [11]*

ML
INTEGRATED

Automatic call tracing

Decorator-based; wraps any Python function and logs inputs, outputs and duration

LLM app tracking

Tracks model calls, prompt versions and output changes across experiments

Feedback & scoring

Attach human or automated scores to any traced call; surfaces regressions

W&B ecosystem integration

Connects natively with experiment tracking, artifact versioning and model monitoring

Limitation: Primarily for ML experimentation; less focused on production agent monitoring

Langfuse

Source: Langfuse Authors. Langfuse Open-Source LLM Observability. langfuse.com, 2024. [12]

SELF
HOSTABLE

Framework-agnostic tracing

Works with any LLM library via SDKs or OpenTelemetry, not tied to one framework

Trace-level cost & latency

Calculates token costs and latency per trace, supports budget analysis

Prompt management

Version-controlled prompts with A/B comparison tied to execution traces

Evaluation & annotation

Human annotation workflows with LLM-as-a-judge evaluation support

Limitation: Smaller community than LangSmith; fewer built-in evaluators; self-hosting requires DevOps effort.

Tool Comparison

Criterion	LangSmith	Arize Phoenix	W&B Weave	Langfuse
Primary focus	LLM chain tracing	LLM + embedding eval	ML experiments	Open LLM tracing
Trace collection	Auto (LangChain)	OTEL-based	Decorator	SDK / OTEL
Framework agnostic	Partial	Yes	Yes	Yes
Visual dashboard	Rich	Rich	Rich	Good
Evaluation support	Built-in	Strong	Scoring	LLM judge
Decision-point debug	Partial	Partial	Limited	Partial
Self-hostable	Cloud	Yes	Cloud	Yes
Open source	No	Yes	No	Yes
Setup complexity	Medium	High	Low	Medium

What Existing Tools Are Missing

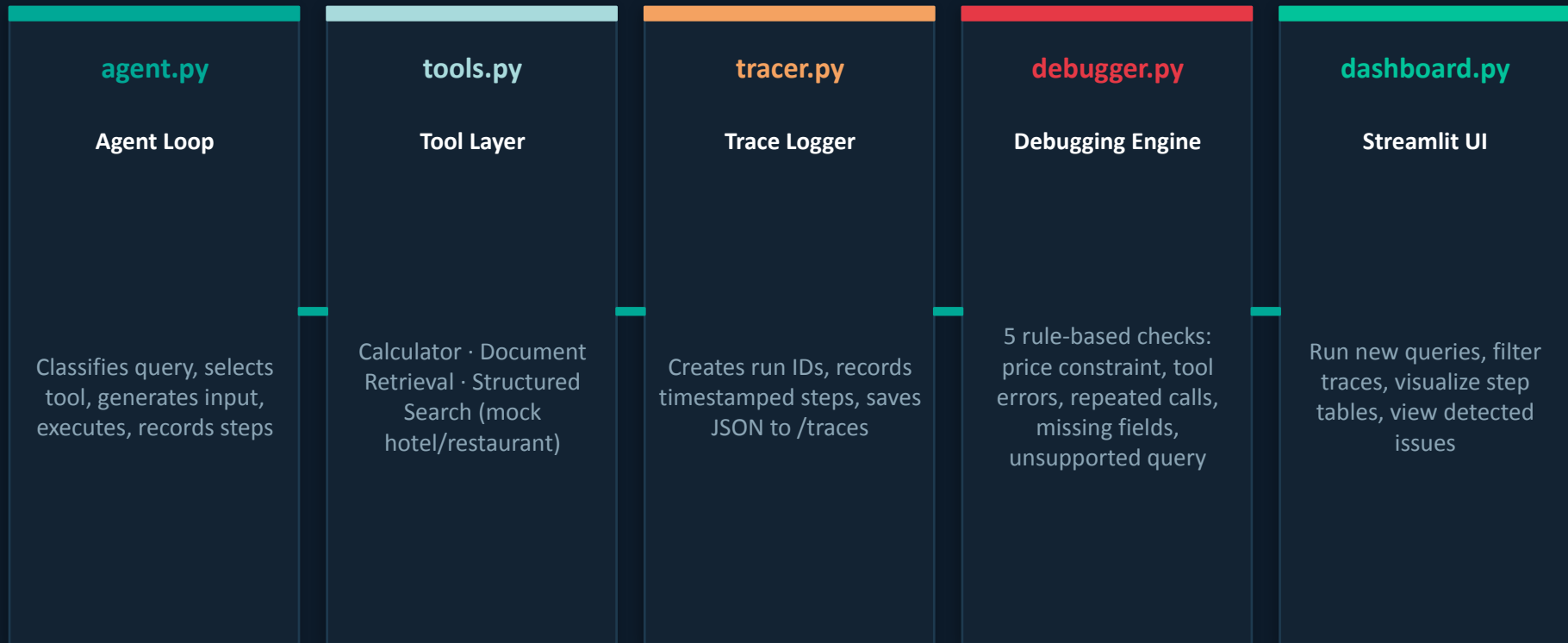
Gaps in Current Tooling

- Cloud-oriented - not built for education
- No decision-point violation detection
- Complex setup - hard to instrument quickly
- Debugging logic not built-in
- Trace structures opaque - hard to teach

Agent Trace Debugger Adds

- Lightweight local setup - zero dependencies
- Rule-based debugging engine built-in
- Step-by-step trajectory visualization
- JSON trace storage for reproducibility
- Explainable by design - easy to teach

Agent Trace Debugger — Architecture



Demo Scenario 1 — Successful Computation

User Query: "What is 25×8 ?"

Step	Event Type	Agent Decision	Tool	Status	Issue
1	user_query	received_user_query	—	✓	—
2	query_analysis	classified_as: computation	—	✓	—
3	tool_selection	select_calculator_tool	calculator_tool	✓	—
4	tool_input_gen	generate_tool_input	calculator_tool	✓	—
5	tool_call	execute_computation	calculator_tool	✓	—
6	final_answer	generate_final_answer	—	✓	—
7	debugging_check	run_debugging_rules	—	✓	none

Result: 200 · Status: SUCCESS · Detected Issues: None

Demo Scenario 2 — Hidden Constraint Violation

User Query: "Find a hotel in Munich under €120."

Step	Event Type	Agent Decision	Detail	Exec	Debug
1	user_query	received_user_query	—	✓	—
2	query_analysis	classified_as: structured_search	—	✓	—
3	tool_selection	select_structured_search_tool	—	✓	—
4	tool_input_gen	generate_tool_input	max_price = 200 ← WRONG	✓	—
5	tool_call	execute_structured_search	max_price = 200	✓	FLAGGED
6	final_answer	generate_final_answer	Hotel A for €110	✓	—
7	debugging_check	run_debugging_rules	price_constraint_mismatch	FAIL	DETECTED

Tool executed successfully. Final answer looks valid. Yet the debugger detected a hidden constraint mismatch at Step 5

Demo Scenario 3 — Missing Structured Input

User Query: "Find a hotel under €120." *(no city specified)*

What happened:

- Query classified as `structured_search`
- No city keyword found
- City defaulted to "Unknown"
- Search returned no match
- Final answer: No matching result
- Debugger flagged: missing city field

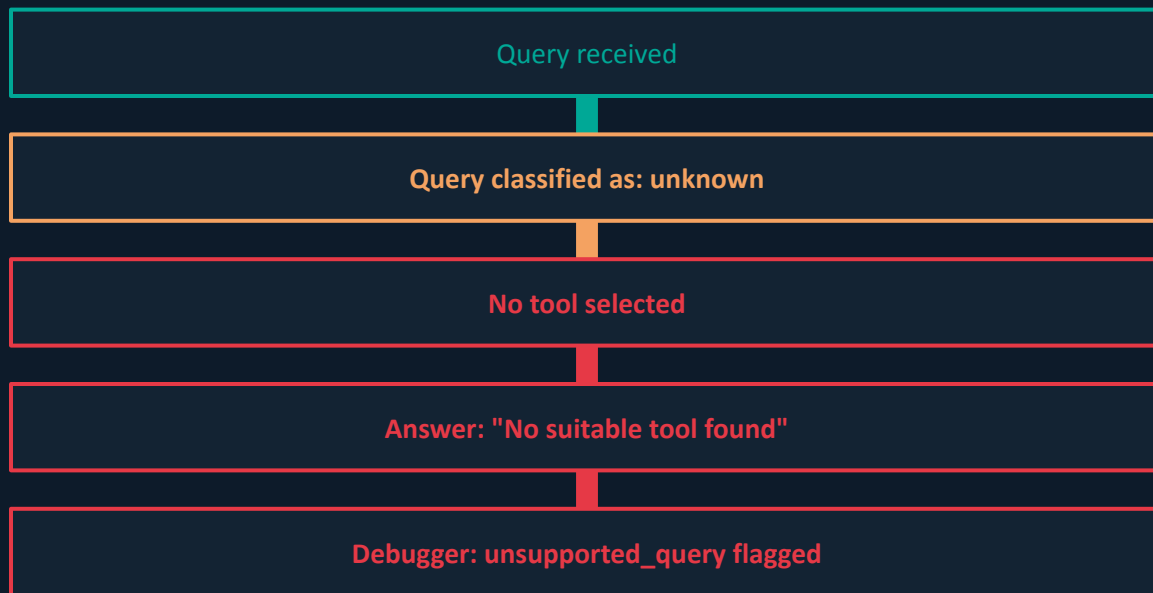
What the trace reveals:

```
event: tool_input_generation
tool_input: {
  "category": "hotel",
  "city": "Unknown",    ← flagged
  "max_price": 120
}

issue: missing_structured_input
field: city
step_id: 5
```

Demo Scenario 4 — Unsupported Query

User Query: "Tell me a joke." *(no matching tool available)*



Why this matters: The agent did not crash. Without tracing, the failure is completely silent.

Interactive Trace Dashboard

Sidebar

- ▶ Run New Query
 - Enter query text
 - Simulate failure
 - [Run Agent]

- ▼ Trace Selection
 - Filter: All / Success / Failed
 - Select run dropdown

Dashboard Summary

Total: 12

Success: 9

Failed: 3

Run Overview: run_679448c3 | Status: FAILED | Steps: 8

User Query: "Find a hotel in Munich under €120."

Final Answer: Hotel A is available in Munich for €110.

⚠ Detected Issue: price_constraint_mismatch at step 5 — User requested max price 120, but tool used max_price=200.

Step Summary Table

step_id	event_type	tool_name	exec_status	debug_status
5	tool_call	structured_search	success	issue_detected

Experimental Results

12

Total Runs

9

Successful

No debugging issues

3

Failed Runs

Issues detected

5

Debug Rules

Rule-based checks

Failure Breakdown

price_constraint_mismatch

1 run

missing_structured_input (city)

1 run

unsupported_query

1 run

Key Observation

All 3 failures were logically silent — tools executed without errors and answers appeared valid. Only the rule-based engine identified the actual problems.

This demonstrates that execution success $\neq$ logical correctness.

Strengths and Limitations

Strengths

- Explainable step-by-step tracing - every decision visible
- Lightweight - Python + Streamlit, no cloud needed
- Detects logical failures invisible to standard tools
- JSON traces enable reproducible experiments
- Rule-based debugger is transparent and easy to extend
- Educational value: ideal for teaching agent observability

Limitations

- Rule-based logic - no real LLM integration
- Mock tools - no external APIs or real-world data
- Small dataset of 12 predefined test queries
- No multi-agent or parallel execution support
- Rule engine does not generalize to arbitrary constraints
- No automated evaluation beyond rule-based checks

Future Work

1

Real LLM Integration

Replace rule-based classification with a live LLM agent to expose non-deterministic failure patterns

2

Advanced Debugging Rules

Extend the rule engine with semantic checks — answer-grounding, contradiction detection, hallucination indicators

3

LLM-as-a-Judge Evaluation

Add an LLM evaluator scoring agent decisions on correctness, relevance and constraint adherence

4

Multi-Agent Tracing

Extend the trace model to support orchestrator–subagent hierarchies and parallel tool invocations

5

Production Integration

Connect to LangSmith or Langfuse via OpenTelemetry to export traces into industrial observability pipelines

6

Failure Analytics Dashboard

Add aggregated failure analytics: failure rate by query type, error categories, latency histograms

Conclusion

- Tool-using agents make multi-step decisions invisible by default — final-output evaluation is not sufficient
- Agent trajectories must be recorded step by step to identify where failures originate
- Existing tools provide production-grade monitoring but lack explicit decision-point violation detection
- The Agent Trace Debugger shows that rule-based trace analysis can surface logically silent failures
- Observability is not a luxury for AI agents — it is a prerequisite for reliable, trustworthy deployment

"Reliable AI agents require visibility into their step-by-step behavior — not only evaluation of their final answers."

References

- [1] B. H. Sigelman et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report, 2010.
- [2] OpenTelemetry Authors. OpenTelemetry Specification. opentelemetry.io, 2024.
- [3] F. Fournier, L. Limonad, Y. David. Agentic AI Process Observability: Discovering Behavioral Variability. [arXiv:2505.20127](https://arxiv.org/abs/2505.20127), 2025.
- [4] Y. Zheng, Y. Hu, T. Yu, A. Quinn. AgentSight: System-Level Observability for AI Agents Using eBPF. [arXiv:2508.02736](https://arxiv.org/abs/2508.02736), 2025.
- [5] W. Epperson et al. Interactive Debugging and Steering of Multi-Agent AI Systems. CHI 2025.
- [6] V. J. Koeman, K. V. Hindriks, C. M. Jonker. Omniscient Debugging for Cognitive Agent Programs. IJCAI 2017.
- [7] D. N. Lam, K. S. Barber. Debugging Agent Behavior in an Implemented Agent System. ProMAS 2004, LNCS 3346.
- [8] D. N. Lam, K. S. Barber. Automated Interpretation of Agent Behaviour. AOIS 2005, LNCS 3529.
- [9] LangChain Inc. LangSmith Documentation. smith.langchain.com, 2024.
- [10] Arize AI. Phoenix Observability Platform Documentation. phoenix.arize.com, 2024.
- [11] Weights & Biases. Weave Documentation. weave.wandb.ai, 2024.
- [12] Langfuse Authors. Langfuse Open-Source LLM Observability. langfuse.com, 2024.

Thank You

Questions & Discussion
